CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)

Assistant
Teaching Professor
Lisa Yan

Parallelism IV: Synchronization

Thoughts about Threads

"Although threads seem to be a small step from sequential computation, in fact, they represent a huge step. They discard the most essential and appealing properties of sequential computation: understandability, predictability, and determinism. **Threads, as a model of computation, are wildly nondeterministic**, and the job of the programmer becomes one of pruning that nondeterminism."

Professor Emeritus
Edward Lee
UC Berkeley

Edward A. Lee. "The Problem with Threads."
Technical Report No. UCB/EECS-2006-1. January 2006.
http://www.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-1.html
See also: Computer 39, 5 (May 2006), 33–42. https://doi.org/10.1109/MC.2006.180

Agenda

Threaded Hello World

- Threaded Hello World
- Data Races
- Critical Sections

— Lower-level implementation —

- Cache Coherence
- Synchronization with Locks

OpenMP API

- **OpenMP** ("Open specifications for Multi-Processing") is an API with language **extensions for C**, C++, and Fortran.
  - Enables **multi-threaded, shared memory parallelism**.
  - Generally follows the fork-join model
  - "Open Multi-Processing"
- Portable, standardized, and easy to compile:
  - `#include <omp.h>`
  - `gcc -fopenmp foo.c`

- Key ideas: (see lab)
  - Shared vs. private variables
  - OpenMP directives for:
    - Parallelization and work sharing
    - Synchronization

Demo

On hive:

```
$ cd lec33_code
$ make
$ ./hello_world              # parallel for loop
$ ./hello_world_add          # data race
```

- `#pragma omp parallel for`
  - Must have relatively simple "shape" for an OpenMP-aware compiler to be able to parallelize it (e.g., how many iterations assigned to thread)
  - No premature exits (break, return, exit, goto)

Thread-Based Parallelism: Assumptions (1/2)

1. Explicit programming model with full programmer control over parallelization

- Pros:
  - Compiler directives are simple and easy to use
  - Legacy serial code does not need to be rewritten
- Cons:
  - Compiler must support OpenMP (e.g. gcc 4.2)
  - Amdahl's law is gonna get you after not too many cores...

Thread-Based Parallelism: Assumptions (2/2)

1. Explicit programming model with full programmer control over parallelization

2. Multiple threads in a **shared memory environment**

- Pros:
  - Takes advantage of shared memory, programmer need not worry (that much) about data placement
- Cons:
  - Code can only be run in shared memory environments
  - **Synchronizing use of shared resources is hard**

Agenda

Data Races

- Threaded Hello World
- Data Races
- Critical Sections

— Lower-level implementation —

- Cache Coherence
- Synchronization with Locks

Nondeterministic Outcomes

[demo]

Suppose we run the below code on 4 threads.

```
int x = 0;
#pragma omp parallel
{
    x = x + 1;
}
```

What are possible values of x after running this code? Select all that apply.

A. 0
B. 1
C. 2
D. 3
E. 4
F. 5 or more

What are possible values of x after running this code? Select all that apply.

Suppose we run the below code on 4 threads.

```
int x = 0;
#pragma omp parallel
{
    x = x + 1;
}
```

A. 0 — 0

B. 1 — 0

C. 2 — 0

D. 3 — 0

E. 4 — 0

F. 5 or more — 0

Three assembly instructions, Interleaved

- For now, assume **atomic instructions** , i.e., no nothing else interposes itself while the instruction is executing.
- Instructions from different threads have their execution on the CPU **interleaved**.

```
// four threads
int x = 0;
#pragma omp parallel
{
    x = x + 1;
}
```

```
sw x0 0(sp)

lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
```

Data Race Case 1: All threads run sequentially

- **Orange** thread load read x: 0
- **Orange** thread store write x: 1
- **Green** thread load read x: 1
- **Green** thread store write x: 2
- **Blue** thread load read x: 2
- **Blue** thread store write x: 3
- **Purple** thread load read x: 3
- **Purple** thread store write x: 4

```
sw x0 0(sp)
load
addi
store
load
addi
store
load
addi
store
load
addi
store
```

Final value of x: 4

Data Race Case 2: Threads perfectly interleaved

- **Orange** thread load read x: 0
- **Green** thread load read x: 0
- **Blue** thread load read x: 0
- **Purple** thread load read x: 0
- **Orange** thread store write x: 1
- **Green** thread store write x: 1
- **Blue** thread store write x: 1
- **Purple** thread store write x: 1

Final value of x: 1

```
sw x0 0(sp)
load
load
load
load
addi
addi
addi
addi
store
store
store
store
```

Agenda

Critical Sections

- Threaded Hello World
- Data Races
- Critical Sections

— Lower-level implementation —

- Cache Coherence
- Synchronization with Locks

Data Race Case 3: Orange Store Happens Last

Orange thread load	read	x: 0
Green thread load	read	x: 0
Green thread store	write	x: 1
Blue thread load	read	x: 1
Blue thread store	write	x: 2
Purple thread load	read	x: 2
Purple thread store	write	x: 3
Orange thread store	write	x: 1

```
sw x0 0(sp)
load
addi
load
addi
store
load
addi
store
load
addi
store
store
```

Final value of x: 1

...etc.

Data Race

- Two memory accesses form a **data race** if:
  - They are from different threads to the same location
  - At least one is a write, and
  - They occur one after another.
- Remember thread model: **shared memory**.
  - For a given thread, these two operations might not happen together:
    - Read current value of x
    - Write new value of x

- Not a data hazard...!
  - Data hazard: Sequential instructions have data dependencies during concurrent execution (instruction-level parallelism via pipelining).
  - Here, even with no ILP, can have **nondeterministic results**. This results from lack of synchronization on which thread accesses memory first.

Critical Sections with OpenMP (1/2)

- A **critical section** is a segment of code that must be executed by a single thread at a time, thereby enforcing **synchronization**.

- `#pragma omp barrier` [docs]
  - Forces all threads to wait until all threads have hit the barrier
- `#pragma omp critical` [docs]
  - Creates a **critical section** within a parallel code segment.
  - Each thread can **safely** execute code in critical section, knowing that it is the only thread that can execute that section at that time
  - This user-level specification (e.g., in OpenMP) relies on hardware synchronization instructions (e.g., in RISC-V. See locks)

Synchronization, Amdahl's Law

- Desired correct outcome:

```
load
addi
store
```

```
load
addi
store
```

```
load
addi
store
```

```
load
addi
store
```

(or any permutation between these four code segments)

- OpenMP has very restrictive parallelism.
  - Really only good for parallelizing loops...
  - Beyond that:
    - If your critical section is too large, then effectively serial program → **Amdahl's law** quickly rears its ugly head
    - If critical sections not defined well, can run into **deadlock**.

Many Demos

$ # update data_race to include critical section
$ # revisit matmul, discuss context switches and why no data races
$ # revisit hello_world with critical sections

No data race with critical section

[demo]

```
1  #include <stdio.h>
2  #include <omp.h>
3  int main() {
4    int x = 0;                          /* shared variable */
5    #pragma omp parallel
6    {
7      #pragma omp critical
8      {
9        x += 1;
10     }
11   }
12 }
13
14
15
16
17
18
19
```

Parallel Hello World, Synchronized [demo]

```
1  #include <stdio.h>
2  #include <omp.h>
3  int main() {
4    int x = 0;                          /* shared variable */
5    #pragma omp parallel
6    {
7      int tid = omp_get_thread_num(); /* private variable */
8      #pragma omp critical
9      {
10       x++;
11       printf("Hello World from thread = %d, x = %d\n", tid, x);
12     }
13     #pragma omp barrier
14     if (tid == 0) {
15       printf("Number of threads = %d\n", omp_get_num_threads());
16     }
17   }
18 }
19
```

DGEMM OpenMP: Why no critical section?

```
#pragma omp parallel for
for (int i = 0; i < A->nrows; i++) {
  for (int j = 0; j < B->ncols; j++) {
    double sum = 0;
    for (int k = 0; k < A->ncols; k++) {
      sum += \
        A->data[i*A->ncols+k]*B->data[k*B->ncols+j];
    }
    C->data[i*C->ncols+j] = sum;
  }
}
```

DGEMM: Notes

[demo notes]

- Consider two threads, one running the i = 0 and i = 1 iteration.
  - Several accesses (read/write) to different levels of memory hierarchy:
    - Sum? Each thread has its own copy of sum. where is this stored? In threads copy of register file
    - Read element A_{ik}: No data race, because different values
    - Read element B_{kj}: No data race, because only read
    - Write element C_{ij}: No data race, because different values

- Operating System performs **context switches**:
  - if thread 0 needs to now write to C, but has cache miss
  - OS: perform context switch to bring in thread 1 for execution
  - OS: store thread 0's regfile, pc to memory (somewhere fast) (this includes sum register)
  - OS: Load in thread 1's regfile, pc to HW regfile, pc
  - Processor on next cycle: Read PC (this is now thread 1's pc)

Context switches are out of scope for this course. Take CS 162!